nticmaster

UNIVERSIDAD COMPLUTENSE MADRID



DOCUMENTACIÓN

MODULE / COURSE
Bases de Datos SQL

TOPIC/UNIT
Bloque 1: Bases de Datos Relacionales

PROFESSOR:
Juan Fernando Sánchez Martínez

1. Unit 1. Introducción a las Bases de Datos

1.1 Bases de Datos Relacionales

1.2 Introducción a SQL

1.3 Estructura de Tablas

2. Unit 2. Modelado Bases de Datos

2.1. Fundamentos del Modelado

2.2. Análisis de Requisitos

2.3. Modelo Conceptual

2.4. Modelo Lógico

2.5. Modelo Físico

3. Unit 3. Gestión de Bases de Datos

3.1. SGBD - Conceptos Generales

3.2. Plataforma Snowflake

3.3. Gestión de Usuarios

3.4. Interfaz de Usuario

1. Introducción a Bases de Datos Relacionales

Conceptos básicos de Bases de Datos

Una **base de datos** es una colección organizada de información o datos que se estructura para facilitar su gestión, almacenamiento y recuperación. Las bases de datos son esenciales en la gestión de información, y su uso se ha extendido ampliamente en diversas áreas, como empresas, organismos gubernamentales, instituciones académicas, y más. La principal función de una base de datos es almacenar información de manera eficiente para que esta pueda ser consultada, modificada y organizada fácilmente.

Existen diferentes tipos de bases de datos, siendo las **bases de datos relacionales** una de las más utilizadas. Estas bases de datos organizan la información en tablas, donde los **datos** se almacenan en forma de **filas** y **columnas**. Las filas representan instancias específicas de datos, mientras que las columnas representan atributos de esos datos. Cada tabla en una base de datos relacional está diseñada para almacenar información sobre una entidad específica, como por ejemplo "Clientes" o "Productos".

El concepto de **relacional** proviene de la capacidad que tienen estas bases de datos de establecer relaciones entre diferentes tablas, conectando datos en una tabla con datos en otra mediante el uso de claves primarias y foráneas. Estas relaciones permiten combinar y consultar datos de manera más flexible y eficiente que en bases de datos tradicionales, como los archivos planos, en los que los datos no están relacionados entre sí.

Los **Sistemas de Gestión de Bases de Datos (SGBD)** son los encargados de gestionar y controlar las operaciones sobre los datos, asegurando que las bases de datos sean accesibles, eficientes y seguras. Un SGBD proporciona las herramientas necesarias para realizar operaciones como inserciones, actualizaciones, eliminaciones y consultas de datos. A través del lenguaje SQL (Structured Query Language), los usuarios interactúan con el SGBD para gestionar los datos sin tener que preocuparse

por los detalles de implementación física de los datos. Algunos de los sistemas de bases de datos relacionales más conocidos son:



MySQL: Un sistema de código abierto que se utiliza principalmente en aplicaciones web. Ofrece una amplia variedad de funcionalidades y es compatible con múltiples lenguajes de programación.

Oracle Database: Un SGBD comercial utilizado en grandes organizaciones que necesitan gestionar grandes volúmenes de datos con alta fiabilidad y rendimiento.

Microsoft SQL Server: El sistema de bases de datos relacionales de Microsoft, ideal para entornos empresariales con un enfoque en la integración con herramientas de Microsoft como .NET y Power BI.

Snowflake: Es una plataforma de gestión de bases de datos en la nube que permite el almacenamiento, análisis y compartición de grandes volúmenes de datos de manera escalable y segura, sin necesidad de infraestructura local.

Ventajas de las Bases de Datos Relacionales

Las bases de datos relacionales ofrecen múltiples ventajas sobre otros tipos de bases de datos, especialmente frente a los archivos planos o sistemas no relacionales. Algunas de las principales ventajas son:

Gestión centralizada mediante un SGBD: Los SGBD facilitan la gestión centralizada de los datos, lo que permite a los usuarios realizar operaciones complejas mediante el

uso de lenguajes de consulta como SQL. Además, centralizar la gestión permite mantener la consistencia y la integridad de los datos.

Capas de abstracción de datos: Uno de los beneficios clave de las bases de datos relacionales es que los usuarios no necesitan preocuparse por cómo están almacenados físicamente los datos. El SGBD oculta los detalles de almacenamiento físico y ofrece diferentes niveles de abstracción, desde el nivel conceptual hasta el nivel físico.

Independencia física y lógica de los datos: Las bases de datos relacionales permiten realizar cambios en la estructura física de los datos (por ejemplo, cambiar el formato o ubicación del almacenamiento) sin afectar a la manera en que los usuarios consultan o actualizan los datos. Esto también se extiende a la lógica de la base de datos, permitiendo cambios en el diseño lógico sin afectar el acceso a los datos.

Acceso concurrente por múltiples usuarios: A diferencia de los archivos planos, las bases de datos relacionales están diseñadas para permitir el acceso simultáneo de múltiples usuarios. Esto es crucial en entornos empresariales donde varias personas deben consultar o modificar los mismos datos al mismo tiempo, sin que haya inconsistencias o bloqueos.

Mayor seguridad y gobernabilidad: Los sistemas de bases de datos relacionales permiten la implementación de políticas de seguridad robustas, asegurando que solo los usuarios autorizados puedan acceder o modificar ciertos datos. Las empresas suelen utilizar esquemas de permisos jerárquicos, donde se asignan distintos niveles de acceso a diferentes usuarios o roles. Por ejemplo, un empleado del departamento de ventas podría tener acceso solo a los registros de clientes, mientras que un administrador de TI tendría acceso completo a todas las tablas y registros. Esto es vital para asegurar la integridad y la confidencialidad de los datos, especialmente en industrias reguladas como la sanitaria o la financiera.

Consistencia de los datos: Gracias a la implementación de reglas de integridad referencial (mediante claves primarias y foráneas), las bases de datos relacionales garantizan que las relaciones entre tablas y registros siempre sean válidas. Esto

significa que, por ejemplo, no se podrá registrar un pedido para un cliente que no exista en la tabla de clientes. Las bases de datos relacionales también permiten la aplicación de reglas personalizadas para asegurar que los datos ingresados cumplan con las normas y formatos establecidos.

Transacciones y control de concurrencia: Una de las características más poderosas de los SGBD es su capacidad para manejar transacciones. Las **transacciones** son un conjunto de operaciones que deben cumplirse todas o ninguna, lo que asegura la consistencia de los datos incluso en situaciones de error. Además, los sistemas relacionales permiten el acceso concurrente de múltiples usuarios sin comprometer la integridad de los datos. El control de concurrencia garantiza que dos usuarios no puedan modificar los mismos datos simultáneamente de una manera que cause inconsistencias o pérdida de datos.

Recuperación ante desastres: Los SGBD también ofrecen opciones avanzadas de recuperación ante fallos o desastres. Mediante técnicas como el registro de transacciones y las copias de seguridad automáticas, es posible restaurar una base de datos a un estado anterior si se produce un fallo en el sistema, un corte de energía o una corrupción de los datos. Este tipo de capacidades son esenciales en entornos donde la pérdida de datos puede tener consecuencias graves, como en sistemas bancarios o de salud.

Lenguaje SQL

El **Structured Query Language (SQL)** es el lenguaje estándar para interactuar con bases de datos relacionales. Introducido en la década de 1970, SQL permite a los usuarios realizar una variedad de operaciones sobre los datos almacenados en una base de datos, que incluyen la **consulta**, **manipulación** y **definición** de datos, así como el control del acceso a estos.

SQL está estructurado en varias categorías que permiten a los usuarios ejecutar diferentes tipos de comandos, dependiendo de la tarea que se desee realizar:

Lenguaje de Definición de Datos (DDL): Esta categoría de SQL se utiliza para definir o modificar la estructura de las bases de datos, incluyendo tablas, índices y vistas. Ejemplos de comandos de DDL incluyen:

CREATE: Permite crear nuevos objetos dentro de la base de datos, como una tabla o un índice.

ALTER: Se utiliza para modificar la estructura de un objeto existente, como agregar o eliminar columnas en una tabla.

DROP: Elimina objetos de la base de datos, como tablas, índices o vistas.

Los comandos de DDL permiten que los administradores de bases de datos creen y gestionen la estructura lógica de la base de datos, configurando cómo se almacenan y organizan los datos.

Lenguaje de Manipulación de Datos (DML): Es el conjunto de comandos que se utiliza para gestionar los datos propiamente dichos, es decir, para realizar operaciones de lectura, inserción, actualización o eliminación de registros. Entre los comandos más comunes están:

SELECT: Es quizás el comando más utilizado en SQL, y permite consultar y recuperar datos de una o varias tablas.

INSERT: Permite añadir nuevos registros a una tabla.

UPDATE: Se utiliza para modificar registros existentes en una tabla.

DELETE: Elimina registros de una tabla.

El lenguaje DML es la herramienta principal de los usuarios finales para manipular los datos. Mediante estos comandos, es posible realizar consultas complejas que integren información de múltiples tablas, aplicar filtros, agrupar datos y ordenar los resultados según criterios específicos.

Lenguaje de Control de Datos (DCL): Se utiliza para gestionar los permisos y el acceso a los datos dentro de la base de datos. Los comandos principales en esta categoría son:

GRANT: Otorga permisos a los usuarios para realizar ciertas operaciones en la base de datos.

REVOKE: Revoca los permisos previamente concedidos a los usuarios.

El DCL asegura que los datos sean accesibles sólo para aquellos que tienen las autorizaciones necesarias, y juega un papel vital en la gestión de la seguridad de la base de datos.

Lenguaje de Control de Transacciones (TCL): Se utiliza para gestionar las transacciones dentro de la base de datos. Los comandos clave son:

COMMIT: Confirma las transacciones, haciendo permanentes los cambios realizados.

ROLLBACK: Revierta las transacciones no confirmadas, devolviendo la base de datos a su estado anterior.

El TCL es fundamental para garantizar la integridad de los datos en entornos multiusuario, permitiendo que las transacciones fallidas puedan revertirse para evitar inconsistencias en la base de datos.

Componentes de una Base de Datos Relacional

Las bases de datos relacionales se componen de varios elementos clave, que trabajan conjuntamente para organizar y gestionar la información. Entre los componentes más importantes están:

Tablas: Son la unidad fundamental de almacenamiento en una base de datos relacional. Cada tabla almacena información sobre una entidad, como "Clientes" o "Pedidos". Las tablas se estructuran en **filas** (registros) y **columnas** (atributos), donde

cada fila representa una instancia de la entidad y cada columna contiene un valor específico relacionado con esa instancia.

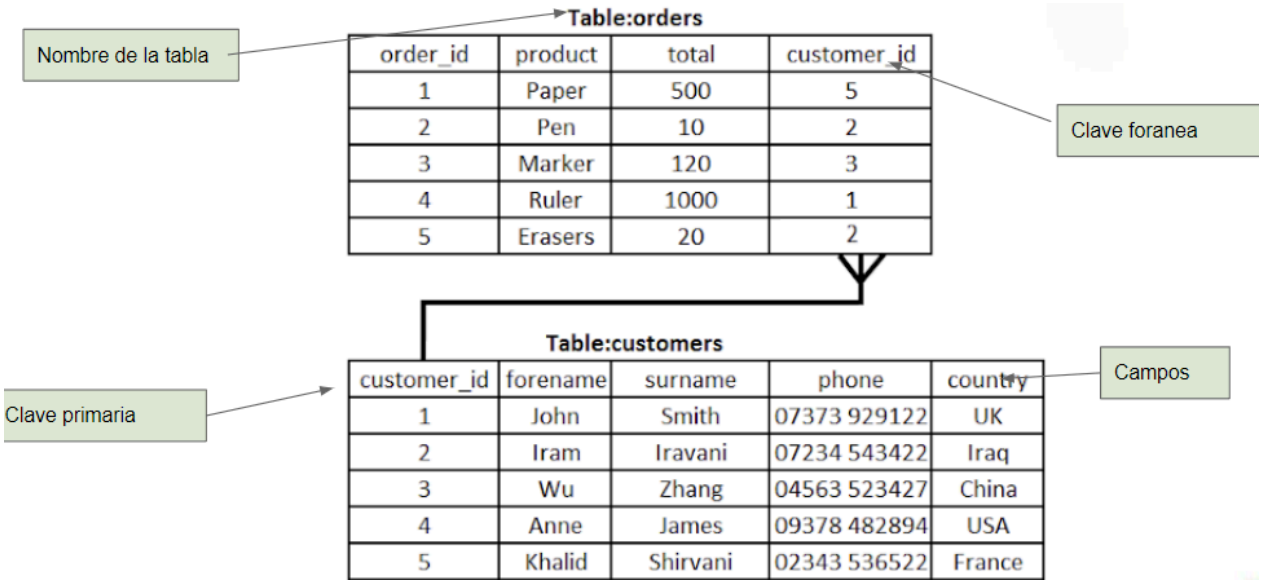
Filas: Cada fila, también conocida como registro o tupla, representa una entrada única en la tabla. Por ejemplo, una fila en la tabla "Clientes" podría contener la información de un cliente individual, como su nombre, dirección, y número de teléfono.

Columnas: Las columnas, también llamadas atributos, definen las propiedades de la entidad representada en la tabla. Cada columna en una tabla tiene un tipo de datos asociado (como texto, número, fecha) que determina el tipo de información que se puede almacenar en esa columna.

Clave Primaria (Primary Key): Es un campo o conjunto de campos dentro de una tabla que identifica de manera única cada registro. La clave primaria es esencial para mantener la unicidad de los datos, asegurando que no haya registros duplicados en una tabla. Además, permite establecer relaciones entre diferentes tablas.

Clave Foránea (Foreign Key): Es un campo en una tabla que establece una relación entre esa tabla y otra. Las claves foráneas son utilizadas para crear vínculos entre tablas, asegurando que los datos estén relacionados de manera coherente. Por ejemplo, en una tabla de "Pedidos", una clave foránea podría referirse a la clave primaria de la tabla "Clientes" para indicar quién realizó el pedido.

Índices: Los índices son estructuras adicionales que se utilizan para mejorar el rendimiento de las consultas. Un índice es una copia ordenada de los datos en una columna o conjunto de columnas, lo que permite al SGBD buscar los registros mucho más rápido. Sin embargo, los índices también ocupan espacio y deben actualizarse cuando se realizan modificaciones en la tabla, por lo que su uso debe estar equilibrado.



2. Modelado de Bases de Datos Relacionales

Diseño Conceptual

El **diseño conceptual** es el primer paso en el proceso de modelado de una base de datos relacional. En esta fase, se desarrollan los primeros esbozos del sistema de información a través de un modelo de alto nivel que no depende de las particularidades del SGBD que se utilizará. El propósito principal del diseño conceptual es representar de manera clara y completa los **requisitos de datos** de la organización, sin preocuparse por los detalles técnicos de implementación.

El enfoque más común para el diseño conceptual es el **Modelo Entidad-Relación (ER)**, propuesto por Peter Chen en 1976. Este modelo permite representar gráficamente las entidades, sus atributos y las relaciones entre ellas. A través de diagramas ER, se puede visualizar la estructura de la base de datos en términos de objetos y sus interacciones, facilitando la comprensión tanto para los diseñadores como para los usuarios no técnicos.

Componentes del Modelo Entidad-Relación:

Entidades: Representan los objetos o conceptos sobre los cuales se desea almacenar información. Cada entidad se modela como una tabla en el diseño lógico. En un

contexto académico, por ejemplo, las entidades podrían ser **Estudiantes**, **Cursos**, o **Profesores**. Las entidades pueden clasificarse como entidades fuertes (independientes) o entidades débiles (dependientes de otras entidades para su existencia).

Atributos: Son las propiedades o características que describen cada entidad. En el caso de la entidad **Estudiantes**, los atributos podrían incluir **Nombre**, **Edad**, **Matrícula**, etc. Los atributos pueden ser simples (no divisibles), compuestos (divisibles en otros atributos) o derivados (cuyo valor puede ser calculado a partir de otros atributos).

Relaciones: Describen las asociaciones entre las entidades. Estas relaciones pueden ser de diferentes cardinalidades, como uno a uno (1:1), uno a muchos (1:N), o muchos a muchos (N:M). Por ejemplo, un **Estudiante** puede estar inscrito en varios **Cursos**, lo que representaría una relación de uno a muchos entre las entidades **Estudiantes** y **Cursos**.

El **Modelo Entidad-Relación (ER)** no solo se limita a describir las entidades y sus atributos, sino que también proporciona una representación detallada de las relaciones entre ellas, permitiendo capturar las reglas del negocio que deben respetarse en la base de datos. En este contexto, los diagramas ER son una herramienta clave que permite visualizar cómo los datos están interrelacionados y cómo fluyen entre las diferentes entidades.

Tipos de Relaciones: Las relaciones en un diagrama ER se clasifican en tres tipos principales según la cardinalidad, que determina cuántas instancias de una entidad pueden asociarse con instancias de otra entidad.

Relaciones Uno a Uno (1:1): En una relación uno a uno, una instancia de una entidad está relacionada únicamente con una instancia de otra entidad y viceversa. Por ejemplo, una persona puede tener asignado un solo pasaporte, y un pasaporte solo puede pertenecer a una persona. En términos de diseño, ambas entidades compartirían una clave común que las vincula de manera exclusiva.
Esta relación no es muy frecuente en bases de datos, pero es útil en ciertos casos,

como cuando se desea dividir una entidad en varias tablas para una mejor organización o por motivos de seguridad, como la separación de datos personales y financieros.

Relaciones Uno a Muchos (1:N): Este tipo de relación es la más común en bases de datos relacionales. En una relación uno a muchos, una instancia de una entidad puede estar relacionada con múltiples instancias de otra entidad, pero las instancias de la segunda entidad solo están relacionadas con una de la primera. Un ejemplo típico es la relación entre **Clientes** y **Pedidos**: un cliente puede hacer varios pedidos, pero cada pedido está relacionado con un solo cliente.
Este tipo de relación se implementa utilizando claves foráneas. En el ejemplo anterior, la tabla **Pedidos** tendría una columna que actúa como clave foránea, haciendo referencia a la clave primaria de la tabla **Clientes**.

Relaciones Muchos a Muchos (N:M): En este tipo de relación, varias instancias de una entidad pueden estar relacionadas con varias instancias de otra entidad. Un ejemplo común es la relación entre **Estudiantes** y **Cursos**: un estudiante puede estar inscrito en varios cursos, y un curso puede tener varios estudiantes. Esta relación no puede implementarse directamente en un modelo relacional sin la creación de una **tabla intermedia** (también llamada tabla de unión o tabla de enlace), que rompe la relación M en dos relaciones 1

La tabla intermedia suele tener como claves foráneas las claves primarias de las dos tablas que vincula. Por ejemplo, la tabla **Inscripciones** en una universidad vincularía la tabla **Estudiantes** y la tabla **Cursos** mediante las claves foráneas **Estudiante_ID** y **Curso_ID**.

Diagramas ER: Los diagramas ER son representaciones visuales que utilizan símbolos gráficos para representar las entidades, atributos y relaciones. Las entidades se representan mediante rectángulos, los atributos mediante elipses, y las relaciones mediante rombos. Estos diagramas facilitan la comunicación entre los diseñadores y los usuarios de la base de datos, ya que proporcionan una visión clara de la estructura del sistema de información.

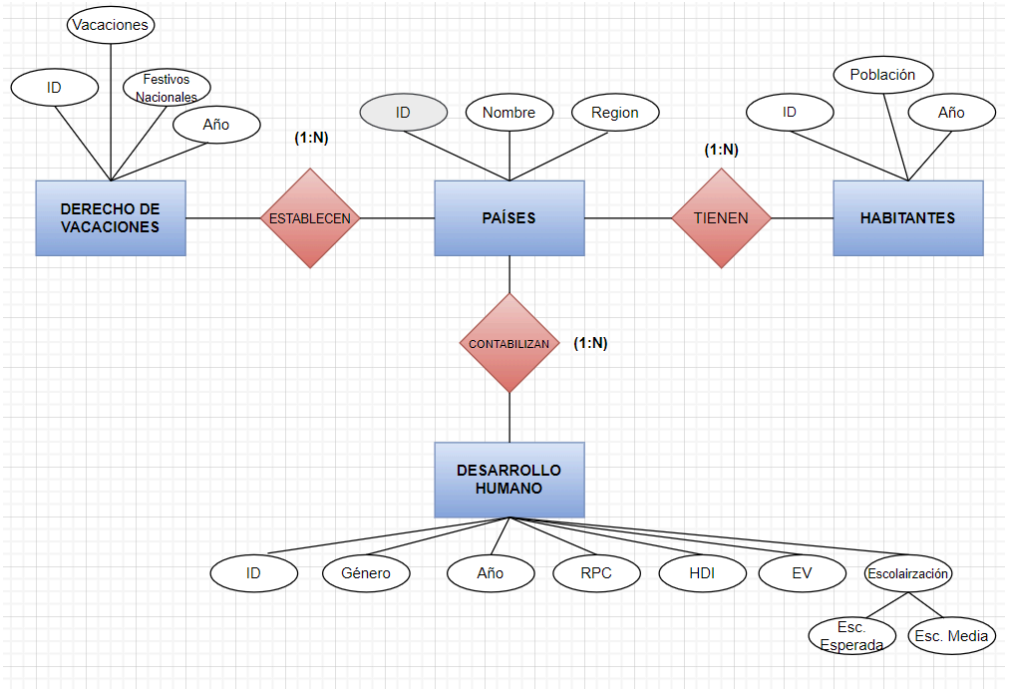
Por ejemplo, un diagrama ER que representa la relación entre estudiantes y cursos incluiría:

Un rectángulo etiquetado como "Estudiantes" con elipses que indican los atributos como "Nombre", "Matrícula", etc.

Otro rectángulo etiquetado como "Cursos" con atributos como "Nombre del curso" y "Código de curso".

Un rombo entre ambas entidades etiquetado como "Inscripción", indicando que los estudiantes se inscriben en cursos. Esta relación puede tener su propio atributo, como "Fecha de inscripción".

Estos diagramas no solo definen la estructura de los datos, sino que también ayudan a identificar y resolver posibles inconsistencias antes de que el modelo se implemente.



Diseño Lógico

Normalización en el Diseño Lógico de Bases de Datos

La normalización es un proceso crítico en el diseño de bases de datos que organiza la información en tablas y relaciones para evitar redundancias y preservar la integridad de los datos. Este proceso se lleva a cabo aplicando una serie de reglas conocidas como formas normales (FN), cada una con un objetivo específico. Estas formas garantizan que la estructura de la base de datos sea eficiente, fácil de mantener y flexible a medida que la cantidad de datos crece.

Primera Forma Normal (1FN)

La primera forma normal (1FN) establece que cada columna en una tabla debe contener solo valores indivisibles, es decir, atómicos. No se permite que una celda almacene listas o conjuntos de valores. Esto significa que la estructura de la tabla debe ser tal que cada registro sea único y cada celda solo contenga un valor específico.

Ejemplo y Explicación: Imagina una tabla de estudiantes que almacena números de teléfono. Si un estudiante tiene varios teléfonos en una sola celda (por ejemplo, "123, 456"), la tabla no estaría en 1FN. Para cumplir con la 1FN, cada teléfono debe tener su propia fila, asegurando así que cada celda sea indivisible. Esto evita complicaciones al consultar datos y mejora la consistencia.

Segunda Forma Normal (2FN)

La segunda forma normal (2FN) tiene como objetivo eliminar las dependencias parciales. Una tabla está en 2FN si cumple con 1FN y, además, todas las columnas no clave dependen completamente de la clave primaria, que puede estar compuesta por una o más columnas. Esto es esencial para evitar que ciertos datos dependan solo de una parte de una clave compuesta.

Ejemplo y Explicación: Considera una tabla que almacena pedidos, donde la clave primaria es una combinación de PedidoID y ProductoID. Si la columna Proveedor depende solo del ProductoID y no de la combinación completa de la clave primaria, es

una dependencia parcial. Para resolverlo, la tabla debe dividirse en dos: una tabla de productos con su proveedor y otra que relacione los pedidos con los productos.

Tercera Forma Normal (3FN)

La tercera forma normal (3FN) elimina las dependencias transitivas. Esto significa que todas las columnas no clave deben depender exclusivamente de la clave primaria, y no de otras columnas no clave. Si una columna depende de otra columna no clave, debe moverse a una tabla separada.

Ejemplo y Explicación: Supongamos que en una tabla de estudiantes tienes las columnas EstudianteID, Nombre, Ciudad y CódigoPostal. Si CódigoPostal depende de Ciudad y no directamente de EstudianteID, entonces existe una dependencia transitiva. Para normalizarla a 3FN, se crea una nueva tabla con Ciudad y CódigoPostal, y la tabla de estudiantes solo incluirá EstudianteID, Nombre y una referencia a Ciudad.

Cuarta Forma Normal (4FN)

La cuarta forma normal (4FN) se enfoca en eliminar las dependencias multivaluadas, que se presentan cuando una celda puede tener múltiples valores relacionados, creando redundancia y complicaciones en el mantenimiento de datos. Una tabla está en 4FN si ya cumple con 3FN y no tiene estas dependencias.

Ejemplo y Explicación: Imagina una tabla donde un profesor puede enseñar varios cursos y estar en diferentes departamentos. Si ambos atributos se almacenan en una sola tabla, esto crea una dependencia multivaluada. La solución es separar estos datos en tablas intermedias que relacionen al profesor con sus cursos y departamentos de manera independiente.

Quinta Forma Normal (5FN)

La quinta forma normal (5FN), también conocida como forma de proyección de unión, asegura que las relaciones complejas puedan descomponerse en tablas más pequeñas sin pérdida de datos. Es un paso adicional que se aplica cuando se trabaja con datos muy relacionados y complejos.

Ejemplo y Explicación: Supongamos una base de datos que registra asociaciones entre productos, proveedores y pedidos. Si existe una relación que puede dividirse en tablas más pequeñas sin perder información y aún así mantener la integridad de los datos, se aplicará la 5FN para garantizar que la estructura de datos sea la más eficiente posible.

Beneficios de la Normalización

La normalización permite representar datos complejos de una manera organizada y eficiente. Algunas de las ventajas más destacadas incluyen:

Reducción de la redundancia de datos, lo que optimiza el uso del espacio y reduce el riesgo de inconsistencias.

Facilidad de mantenimiento, al permitir actualizaciones y modificaciones con menor riesgo de errores.
Integridad referencial, garantizando que las relaciones entre datos se mantengan correctas.

Si bien la denormalización (hacer lo opuesto a la normalización) puede ser útil en algunos casos específicos para mejorar el rendimiento de las consultas, la normalización sigue siendo una práctica esencial para asegurar la estabilidad y la calidad de las bases de datos en la mayoría de los sistemas.

Desarrollo del Diseño Lógico

Una vez realizada la normalización se pasa al **diseño lógico** que es la segunda etapa en el proceso de modelado de una base de datos relacional. En esta fase, el modelo conceptual se traduce a un modelo lógico que puede implementarse en un **Sistema de Gestión de Bases de Datos Relacional (SGBD)**. Aquí, las entidades, atributos y relaciones identificadas durante el diseño conceptual se transforman en tablas, columnas y claves que definen cómo se estructurará y organizará la base de datos.

El propósito principal del diseño lógico es asegurar que los datos estén organizados de manera eficiente y que las relaciones entre las entidades sean adecuadas para las

necesidades de la aplicación. Durante esta fase, los diseñadores toman decisiones clave sobre cómo los datos serán almacenados y gestionados.

Componentes del Diseño Lógico:

Tablas: Cada entidad del diseño conceptual se convierte en una tabla en el diseño lógico. Las tablas contienen columnas (atributos) y filas (registros), y cada fila representa una instancia específica de la entidad. Por ejemplo, si en el modelo conceptual se definió una entidad **Clientes**, en el diseño lógico se creará una tabla **Clientes** con columnas que correspondan a los atributos definidos en el modelo conceptual, como **Nombre**, **Dirección**, **Teléfono**, etc.

Claves Primarias: Se definen claves primarias para cada tabla. Una **clave primaria** es un atributo o un conjunto de atributos que identifica de manera única a cada registro dentro de la tabla. La clave primaria es esencial para mantener la integridad de los datos y evitar la duplicación. Además, las claves primarias permiten establecer relaciones entre diferentes tablas.
Es importante que las claves primarias sean atributos que no cambien con el tiempo, como un número de identificación o un código de producto, ya que cualquier cambio en la clave primaria afectaría las relaciones entre las tablas.

Claves Foráneas: Las **claves foráneas** se utilizan para establecer relaciones entre tablas. Una clave foránea es un atributo en una tabla que referencia la clave primaria de otra tabla. Esto permite crear relaciones como uno a muchos o muchos a muchos. Por ejemplo, en una base de datos de un sistema de compras, la tabla **Pedidos** podría tener una clave foránea llamada **Cliente_ID**, que hace referencia a la clave primaria **ID_Cliente** de la tabla **Clientes**.
El uso correcto de claves foráneas es crucial para mantener la integridad referencial de la base de datos, lo que significa que no se podrán agregar registros en una tabla que no tengan una correspondencia válida en la tabla referenciada. En el ejemplo anterior, no sería posible crear un pedido si no existe un cliente asociado.

Normalización: Durante el diseño lógico, también se aplica el proceso de **normalización**, que es un conjunto de reglas que ayudan a organizar los datos para

minimizar la redundancia y evitar inconsistencias. Las reglas de normalización se estructuran en **formas normales**, siendo la más común la tercera forma normal (3FN). Este proceso implica dividir las tablas que contienen datos redundantes en tablas más pequeñas y relacionadas.
Por ejemplo, en lugar de almacenar el nombre de un proveedor en múltiples registros de la tabla **Pedidos**, sería más eficiente crear una tabla **Proveedores** donde cada proveedor esté representado solo una vez, y luego hacer referencia a esa tabla desde la tabla **Pedidos** mediante una clave foránea.

Tipos de Datos: Se seleccionan los tipos de datos apropiados para cada columna en las tablas. Los tipos de datos determinan qué tipo de información se puede almacenar en una columna y garantizan que los datos se almacenen de manera eficiente. Los tipos de datos más comunes incluyen:

Enteros: Para números enteros, como identificadores únicos o cantidades.

Cadenas de texto: Para almacenar texto variable, como nombres o descripciones.

Fechas: Para registrar fechas y horas, como la fecha de creación de un pedido o la fecha de inscripción de un estudiante.

La elección de los tipos de datos es crucial para el rendimiento de la base de datos, ya que algunos tipos de datos ocupan más espacio que otros. Seleccionar el tipo de dato adecuado para cada columna puede mejorar tanto la eficiencia en el almacenamiento como el tiempo de respuesta de las consultas.

Ejemplo de Diseño Lógico: En un sistema de gestión universitaria, el diseño lógico para la entidad **Estudiantes** podría traducirse a la siguiente tabla:

Tabla Estudiantes:

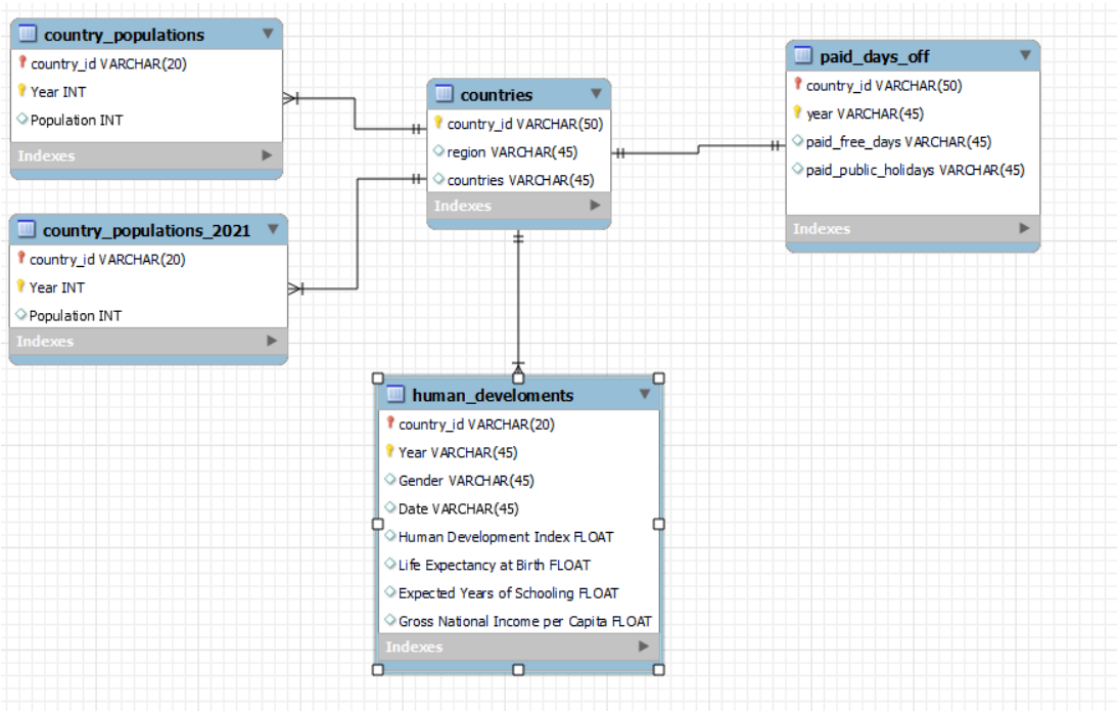
ID_Estudiante (Clave primaria, Entero)

Nombre (Cadena de texto)

Fecha de nacimiento (Fecha)

Correo electrónico (Cadena de texto)

Por otro lado, la entidad **Cursos** se convertiría en la tabla **Cursos**, y la relación entre **Estudiantes** y **Cursos** se modelaría a través de una tabla intermedia llamada **Inscripciones**, que contendría claves foráneas de ambas tablas.



Diseño Físico

El **diseño físico** es la fase final en el modelado de una base de datos relacional. Aquí, el modelo lógico se convierte en un conjunto de instrucciones que pueden implementarse en un SGBD específico, como MySQL, SQL Server, PostgreSQL, entre otros. En esta etapa, los diseñadores se enfocan en aspectos como el rendimiento, el almacenamiento, y la manera en que los datos serán gestionados y recuperados físicamente.

Mientras que el diseño lógico se enfoca en la organización de los datos, el diseño físico se centra en cómo se almacenarán esos datos en discos, cómo se optimizará su acceso, y cómo se protegerán contra posibles fallos del sistema.

Aspectos clave del diseño físico:

Índices: Los **índices** son estructuras que permiten mejorar el rendimiento de las consultas al acelerar la búsqueda de registros en grandes tablas. Un índice es similar al índice de un libro: en lugar de recorrer todas las páginas para encontrar una palabra, el índice te lleva directamente al lugar donde se menciona esa palabra. En una base de datos, los índices se crean en una o más columnas de una tabla para facilitar la búsqueda de registros específicos.
Aunque los índices mejoran el rendimiento de las consultas, también ocupan espacio en disco y pueden ralentizar las operaciones de inserción, actualización y eliminación, ya que el índice también debe actualizarse cada vez que cambian los datos. Por esta razón, es importante equilibrar el uso de índices y seleccionar cuidadosamente en qué columnas se deben crear.

Particionamiento de Tablas: En bases de datos con grandes volúmenes de datos, el **particionamiento** es una técnica que mejora el rendimiento al dividir una tabla grande en varias tablas más pequeñas (particiones). Cada partición contiene un subconjunto de los datos, y las consultas se ejecutan solo en la partición relevante, lo que reduce significativamente el tiempo de respuesta.
El particionamiento puede ser horizontal (dividiendo las filas en varias tablas según algún criterio, como la fecha) o vertical (dividiendo las columnas en diferentes tablas). Por ejemplo, en una tabla de pedidos con millones de registros, se podría crear una partición por año, lo que permitiría que las consultas que solo involucren pedidos recientes se ejecuten más rápido.

Almacenamiento y Recuperación de Datos: El diseño físico también implica tomar decisiones sobre cómo se almacenarán los datos en disco. Esto incluye la organización de los datos en páginas de disco, la gestión de la fragmentación, y la planificación de la recuperación ante fallos. Los sistemas modernos de bases de datos ofrecen opciones

avanzadas para la recuperación de datos en caso de fallos, como el uso de archivos de log de transacciones, que permiten restaurar la base de datos a un estado coherente después de un fallo.

Tamaño de los Bloques de Datos: El tamaño de los bloques de datos (o páginas de datos) es otra consideración importante en el diseño físico. Los bloques de datos son las unidades de almacenamiento que utilizan los SGBD para leer y escribir datos en el disco. Un tamaño de bloque adecuado puede mejorar la eficiencia del acceso a los datos, especialmente en sistemas que manejan grandes volúmenes de transacciones o consultas complejas.

```
create or replace TABLE ONU.HUMAN_STUDY.HUMAN_STUDY (
    COUNTRY_ID VARCHAR(16777216),
    COUNTRY VARCHAR(16777216),
    GENDER VARCHAR(16777216),
    YEAR NUMBER(38,0),
    DATE DATE,
    HUMAN_DEVELOPMENT_INDEX NUMBER(38,9),
    LIFE_EXPECTANCY_AT_BIRTH NUMBER(38,4),
    EXPECTED_YEARS_OF_SCHOOLING NUMBER(38,9),
    MEAN_YEARS_OF_SCHOOLING NUMBER(38,9),
    GROSS_NATIONAL_INCOME_PER_CAPITA NUMBER(38,7)
);
```

Show less

3. Gestión de BBDD con Snowflake



Snowflake es una plataforma de gestión de bases de datos en la nube que integra capacidades de almacén de datos, lago de datos y motor de análisis avanzado. Su arquitectura elástica y escalable permite almacenar y analizar grandes volúmenes de datos sin requerir infraestructura local, lo cual lo hace ideal para organizaciones que buscan simplificar la administración de infraestructura y enfocarse en el análisis de datos.

Arquitectura de Snowflake

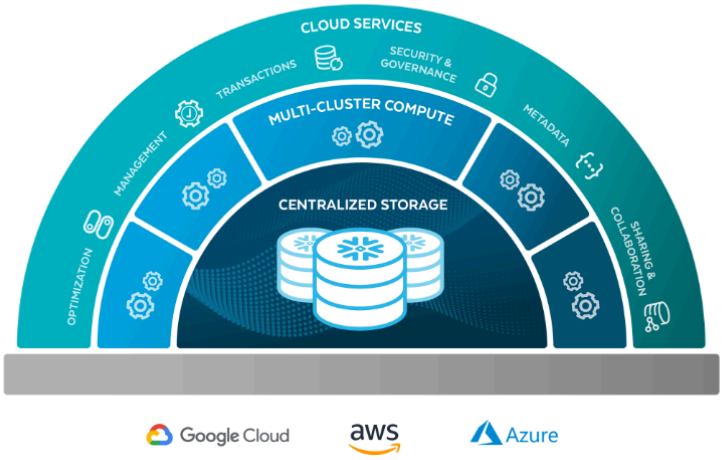
Snowflake está diseñado sobre una arquitectura de almacenamiento y cómputo separada, específica para la nube. Los principales componentes de esta arquitectura son:

Almacenamiento de Datos: Snowflake organiza los datos en "micro-particiones", que son bloques comprimidos de datos. Cada micro-partición contiene un subconjunto de filas y columnas, lo cual permite un almacenamiento eficiente y acceso rápido. La compresión avanzada reduce costos de almacenamiento, y los datos se distribuyen para optimizar el rendimiento de las consultas.

Capa de Servicios: Esta capa incluye servicios que gestionan transacciones, autenticación, control de acceso y optimización de consultas. También maneja la seguridad de los datos, las configuraciones de usuario y otras tareas administrativas, permitiendo que los usuarios realicen consultas complejas sin preocuparse por la infraestructura subyacente.

Cómputo Escalable: Snowflake utiliza "Virtual Warehouses" (almacenes virtuales), que son grupos de recursos de cómputo que ejecutan consultas SQL. Cada almacén virtual es independiente y puede escalarse según la demanda. Además, se pueden suspender automáticamente para ahorrar costos cuando no se utilizan.

SNOWFLAKE ARCHITECTURE



Características Clave de Snowflake

Elasticidad y Escalabilidad Dinámica: Snowflake permite escalar dinámicamente los recursos de cómputo según las necesidades de la carga de trabajo. Esto resulta útil para manejar grandes volúmenes de datos, cargas irregulares y asegurar un rendimiento sin tiempos de espera.

Seguridad Integral: Snowflake prioriza la seguridad mediante:

> **Cifrado Completo**: Los datos se cifran tanto en reposo como en tránsito.
>
> **Control de Acceso Basado en Roles (RBAC)**: Define roles específicos con permisos detallados, por ejemplo, acceso de solo lectura para analistas y acceso completo para administradores.
>
> **Auditoría y Cumplimiento Normativo**: Cumple con normativas como SOC 2 Type II, HIPAA y GDPR, y permite auditar las actividades de los usuarios.

Multi-Cloud y Múltiples Regiones: Snowflake está disponible en las principales nubes (AWS, Azure y Google Cloud), lo cual permite a las organizaciones operar y replicar datos entre nubes para asegurar alta disponibilidad y recuperación ante desastres.

Data Sharing: Con la función **Secure Data Sharing**, Snowflake permite compartir datos en tiempo real sin moverlos físicamente, lo cual mejora la seguridad y eficiencia en comparación con soluciones tradicionales.

Soporte para Datos Semi-Estructurados: Snowflake permite almacenar y consultar datos semi-estructurados como JSON, Avro y XML sin necesidad de transformaciones previas, utilizando la función **VARIANT** para integrar estos datos con los relacionales.

Carga y Consulta de Datos

Carga de Datos: Snowflake facilita la carga de datos desde distintas fuentes con el comando **COPY INTO**, que transfiere datos desde almacenamiento en la nube (como AWS S3 o Azure Blob Storage) a tablas en Snowflake. Este proceso se puede paralelizar para mejorar el rendimiento.

Consultas Optimizadas: Snowflake usa una técnica llamada **Pruning** para limitar la cantidad de micro-particiones requeridas en una consulta, acelerando el tiempo de respuesta. Además, paraleliza las consultas mediante múltiples nodos de cómputo, haciendo el procesamiento más eficiente incluso para grandes volúmenes de datos.

Casos de Uso de Snowflake

Inteligencia de Negocio y Análisis Predictivo: Snowflake es ideal como almacén de datos centralizado que consolida la información relevante para el negocio. Herramientas de Business Intelligence (BI) como Tableau, Power BI o Looker pueden utilizar Snowflake para desarrollar modelos predictivos y realizar análisis avanzados.

Integración de Datos Multi-Fuente: Snowflake facilita la integración de datos desde múltiples fuentes, como aplicaciones de marketing, ERP y CRM. Soporta tanto datos relacionales como semi-estructurados, ofreciendo una visión completa de la empresa sin complicaciones.

Cargas de Trabajo Complejas: La capacidad de escalado automático de Snowflake lo hace adecuado para cargas de trabajo intensivas en análisis, tanto para procesamiento por lotes como consultas ad-hoc, sin afectar el rendimiento.

Recuperación ante Desastres y Alta Disponibilidad

Replicación de Datos: Snowflake permite la replicación automática de bases de datos entre diferentes cuentas, regiones e incluso nubes, garantizando la disponibilidad de los datos ante fallos.

Time Travel y Fail-Safe: Snowflake tiene una función de **Time Travel** que permite acceder a versiones anteriores de los datos por hasta 90 días. Además, la función **Fail-Safe** asegura una copia de los datos durante 7 días adicionales para mayor seguridad.

3.6 Integración y Compatibilidad

Compatibilidad con Herramientas de BI y ETL: Snowflake se integra fácilmente con herramientas ETL como Talend, Informatica y Matillion, y con herramientas de visualización de datos como Tableau, Looker y Power BI, facilitando la integración con el ecosistema de datos existente.

Conectividad SQL: Snowflake ofrece conectores JDBC/ODBC, lo cual facilita su integración con casi cualquier aplicación que utilice SQL.

REFERENCIAS BIBLIOGRÁFICAS

1. Amazon AWS. (2023). ¿Qué es una base de datos relacional? Recuperado de https://aws.amazon.com/es/relational-database/

2. Amazon AWS. (2023). ¿Qué es SQL (lenguaje de consulta estructurada)? Recuperado de https://aws.amazon.com/es/what-is/sql/

3. Despujol Zabala, I. (2022). SQL Bases de datos relacionales. YouTube. Recuperado de https://youtube.com/watch?v=1i0DJm3Dr_Y

4. Google Cloud. (2023). ¿Qué es una base de datos relacional (RDBMS)? Recuperado de https://cloud.google.com/learn/what-is-a-relational-database?hl=es-419

5. IBM. (2021). Modelado dimensional: Claves primarias y foráneas. Recuperado de https://www.ibm.com/docs/es/ida/9.1.2?topic=entities-primary-foreign-keys

6. KeepCoding. (2023). Llaves primarias y foráneas de las BBDD relacionales. Recuperado de https://keepcoding.io/blog/llaves-primarias-y-foraneas/

7. Learn Microsoft. (2023). Definir relaciones entre tablas mediante Access SQL. Recuperado de https://learn.microsoft.com/es-es/office/vba/access/concepts/structured-query-language/define-relationships-between-tables-using-access-sql

8. Microsoft Learn. (2023). ¿Qué es una base de datos relacional? Recuperado de https://docs.microsoft.com/es-es/learn/modules/intro-to-relational-databases/

9. QGIS Documentation. (2023). 15.1. Introducción a Bases de Datos Lesson. Recuperado de https://docs.qgis.org/2.0/es/docs/training_manual/database_concepts/db_intro.html

10. Styde.net. (2020). Relaciones entre tablas de bases de datos. Recuperado de https://styde.net/relaciones-entre-tablas-de-bases-de-datos/

11. Aprender Big Data. (2023). Bases de Datos Relacionales y SQL - Introducción. Recuperado de https://aprenderbigdata.com/bases-de-datos-relacionales/

12. DataScientest. (2021). SQL - todo lo que necesitas saber sobre el lenguaje de programación de bases de datos. Recuperado de https://datascientest.com/es/sql-todo-lo-que-necesitas-saber-sobre-el-lenguaje-de-programacion-de-bases-de-datos

13. Universidad Politécnica de Valencia. (2023). SQL Bases de datos relacionales | 2/25 | UPV. Recuperado de https://youtube.com/watch?v=1i0DJm3Dr_Y

14. Universidad Politécnica de Valencia. (2023). 15.1. Introducción a Bases de Datos Lesson. Recuperado de https://docs.qgis.org/2.0/es/docs/training_manual/database_concepts/db_intro.html

15. Universidad Politécnica de Valencia. (2023). ¿Qué es una base de datos relacional? Recuperado de https://aws.amazon.com/es/relational-database/